

SOFTWARE FAILURE DIAGNOSIS AI

A Structured Hypothesis-Based System for Diagnosing Software Failures Under Uncertainty

Project Report

Submitted for:

TECHgium / LTTS Challenge #2

Project Title:

Software Failure Diagnosis AI

Submitted by:

Nisha R S

Department:

Computer Science and Engineering

Institution:

Amity University Bengaluru

Academic Year:

2026–2027

ABSTRACT

Software failures can have multiple possible causes, making diagnosis difficult and uncertain. **Software Failure Diagnosis AI** is a structured system that generates multiple hypotheses, evaluates supporting and contradicting evidence, and calculates confidence for each possible cause.

The system can select a likely cause, recommend a discriminating test, combine compatible hypotheses, or abstain when the available evidence is insufficient. It also uses an investigation budget and stopping mechanism to control the diagnostic process.

The system was evaluated using test and held-out cases, demonstrating the complete workflow from hypothesis generation to final diagnosis.

1. INTRODUCTION

Software failures can occur due to different reasons such as database problems, recent software deployments, server resource issues, external service failures, or configuration errors. Identifying the actual cause can be difficult when multiple explanations are possible.

Software Failure Diagnosis AI is developed to assist in diagnosing such failures using a structured hypothesis-based approach. The system generates multiple possible causes, evaluates available evidence, and assigns confidence to each hypothesis instead of immediately selecting a single cause.

The system can select a supported hypothesis, recommend a diagnostic test when causes are difficult to distinguish, combine compatible causes, or abstain when there is not enough evidence. This approach helps provide a more structured and explainable method for software failure diagnosis.

2. PROBLEM STATEMENT

2.1 Problem Definition

Given a software failure description and a collection of observed evidence, the objective is to identify plausible underlying causes while managing uncertainty and avoiding premature conclusions.

The system should not simply select the first hypothesis that matches an observed symptom.

Instead, it should:

1. Generate multiple alternative hypotheses.
2. Ensure that the hypotheses are meaningfully different.
3. Represent assumptions and predicted effects.
4. Evaluate available evidence against each hypothesis.
5. Distinguish supporting, contradicting, missing, and dependent evidence.
6. Calculate confidence scores.
- 7.
8. Select a sufficiently supported hypothesis when appropriate.
9. Recommend a discriminating test when competing hypotheses remain difficult to distinguish.
10. Combine compatible hypotheses when multiple causes may contribute.
11. Abstain when evidence is insufficient.
12. Operate within a defined investigation budget.

2.2 Input

The system accepts:

- Software failure description.
- Observed evidence.
- Evidence reliability.
- Evidence source/provenance information.

2.3 Output

The system can produce four major diagnostic outcomes:

SELECT

A sufficiently supported hypothesis is selected.

COMBINE

Two compatible hypotheses with sufficient supporting evidence are combined into a joint explanation.

TEST

The leading hypotheses remain difficult to distinguish, so a discriminating diagnostic test is recommended.

ABSTAIN / UNKNOWN

Available evidence is insufficient to confidently identify a known cause.

3. OBJECTIVES

The major objectives of Software Failure Diagnosis AI are:

Objective 1 — Multi-Hypothesis Generation

Generate multiple possible explanations for a software failure instead of immediately selecting one cause.

Objective 2 — Hypothesis Distinctness

Ensure that generated hypotheses represent genuinely different diagnostic possibilities.

Objective 3 — Evidence-Based Reasoning

Evaluate evidence against individual hypotheses and classify the evidence appropriately.

Objective 4 — Confidence Estimation

Calculate confidence using supporting and contradicting evidence while considering evidence reliability.

Objective 5 — Uncertainty Handling

Allow the system to abstain when evidence is insufficient.

Objective 6 — Diagnostic Test Recommendation

When two hypotheses remain close, recommend an additional test that can distinguish between them.

Objective 7 — Hypothesis Synthesis

Allow compatible hypotheses to be combined when multiple causes may jointly contribute to a failure.

Objective 8 — Controlled Investigation

Use an investigation budget and stopping rule to prevent unlimited exploration.

Objective 9 — Evaluation

Evaluate the diagnostic system using both normal test cases and held-out cases.

4. EXISTING SYSTEM AND PROPOSED SYSTEM

4.1 Existing Approach

A simple failure-diagnosis approach can be represented as:

Failure



Match symptom



Select one cause



Return diagnosis

Such an approach may have several limitations.

Premature Selection

The system may select one cause before sufficiently considering alternatives.

Lack of Evidence Classification

Evidence may simply be treated as a matching keyword without distinguishing supporting and contradicting observations.

Double Counting

Multiple observations from the same source may artificially increase confidence.

No Explicit Unknown State

A system may force an available label even when none of the known causes adequately explains the failure.

Limited Diagnostic Guidance

A prediction alone does not necessarily tell an engineer what test should be performed next.

4.2 Proposed System

The proposed system uses a structured reasoning pipeline:

Software Failure



Hypothesis Generation



Hypothesis Validation



Evidence Assessment



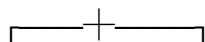
Confidence Scoring



Stopping Analysis



Decision



SELECT COMBINE TEST



ABSTAIN/

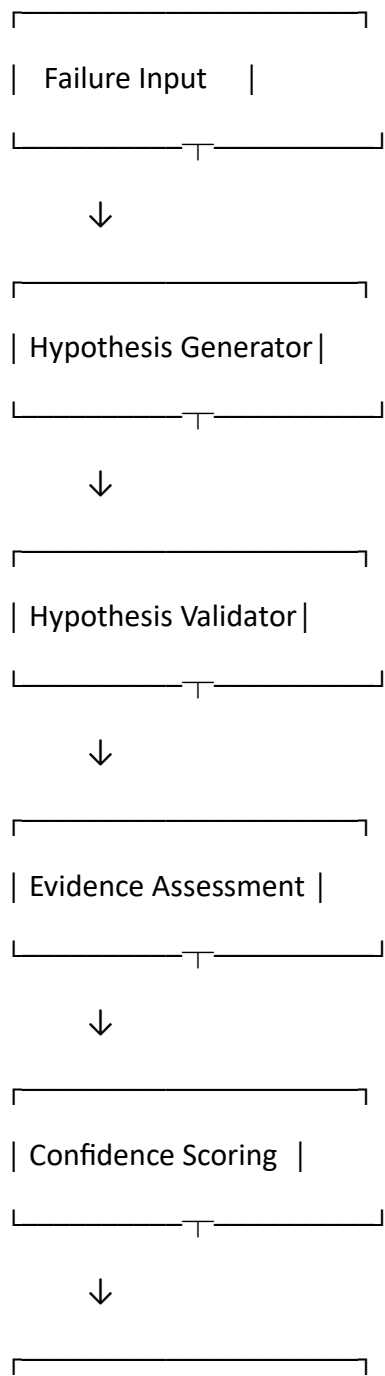
UNKNOWN

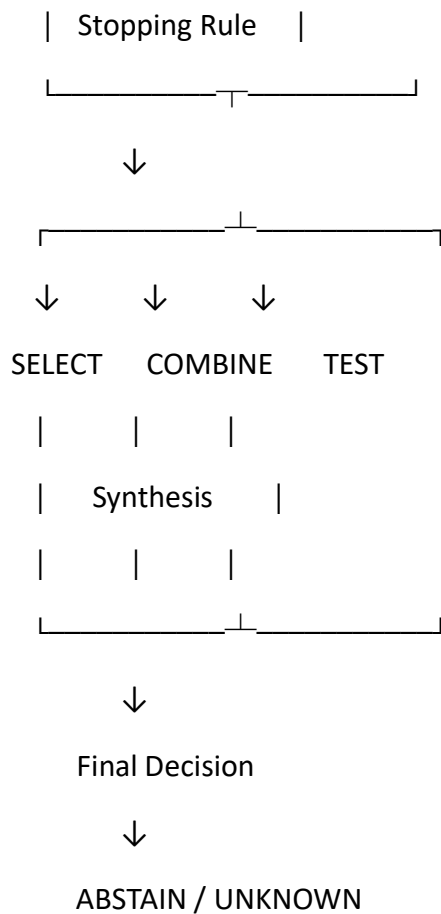
The proposed approach explicitly represents uncertainty and competing explanations.
It also allows diagnostic reasoning to continue only when necessary.

5. SYSTEM ARCHITECTURE

5.1 Architecture Overview

The system is divided into modular components.





5.2 Major Components

Hypothesis Generator

Generates candidate explanations for the observed failure.

The current local diagnostic generator considers:

- H1 — Database failure
- H2 — Recent software deployment
- H3 — Server/resource problem
- H4 — External API failure
- H5 — Configuration/environment problem
- UNKNOWN — Insufficient evidence

Hypothesis Validator

Checks whether each hypothesis contains the required fields and filters invalid or duplicate hypotheses.

Distinctness Engine

Compares hypotheses across multiple axes:

- Cause
- Assumptions
- Predicted effects
- Required evidence

Evidence Assessment Module

Classifies evidence as:

- Supporting
- Contradicting
- Missing
- Dependent

Scoring Module

Calculates raw scores and confidence values using evidence reliability.

Stopping Module

Determines whether the investigation should continue.

Decision Module

Produces SELECT, COMBINE, TEST, or ABSTAIN outcomes.

Test Recommender

Generates a diagnostic test for close competing hypotheses.

Synthesis Module

Combines compatible hypotheses when both have sufficient positive evidence.

Budget Module

Controls the maximum amount of investigation.

6. HYPOTHESIS GENERATION AND DIVERGENCE

6.1 Structured Hypotheses

Each hypothesis is represented using:

Hypothesis

└— ID

└— Cause

└— Assumptions

└— Predicted Effects

└— Required Evidence

For example:

H1 — Database Failure

Cause:

Database failure

Assumptions:

- The application depends on the affected database.
- The database may be unavailable or experiencing connection problems.

Predicted Effects:

- Database connection errors.
- Query failures.
- Connection timeouts.

Required Evidence:

- Database health information.
- Connection error logs.
- Database availability information.

6.2 Current Hypothesis Space

ID	Hypothesis
H1	Database failure
H2	Recent software deployment
H3	Server/resource problem
H4	External API failure

ID	Hypothesis
H5	Configuration/environment problem
UNKNOWN	Insufficient evidence

6.3 Hypothesis Divergence

The system avoids generating multiple hypotheses that express essentially the same cause using different wording.

For example:

H1 → Database failure

H2 → Recent deployment

H3 → Server/resource problem

represent different diagnostic directions.

The purpose of divergence is to maintain a meaningful set of alternatives.

6.4 Multi-Axis Distinctness

Two hypotheses are compared using four axes:

1. Cause
2. Assumptions
3. Predicted effects
4. Required evidence

The conceptual distinctness calculation is:

Distinctness =

(Cause + Assumptions +
Predicted Effects + Required Evidence) / 4

A minimum distinctness threshold is used during validation.

The current implementation uses:

MIN_DISTINCTNESS_SCORE = 0.50

This prevents near-duplicate hypotheses from entering the main diagnostic pipeline.

7. EVIDENCE AND SCORING

7.1 Evidence Representation

Each evidence item contains:

- Evidence ID
- Description
- Source
- Source ID
- Reliability

This allows the system to retain evidence provenance.

7.2 Evidence Types

Supporting Evidence

Evidence that increases confidence in a hypothesis.

Example:

Database connection timeout detected

supports a database-failure hypothesis.

Contradicting Evidence

Evidence that decreases confidence in a hypothesis.

Example:

Database health check is normal

can contradict a database-failure hypothesis.

Missing Evidence

Information that would be useful but is not currently available.

Dependent Evidence

Evidence originating from the same source as another observation.

Dependent evidence should not be counted as independent confirmation.

7.3 Evidence Reliability

Each evidence item has a reliability value.

Higher reliability contributes more strongly to the score.

Conceptually:

High reliability



Greater score contribution

Low reliability



Smaller score contribution

7.4 Raw Score

The system calculates:

Raw Score =

Supporting Weight - Contradicting Weight

where:

Supporting Weight =

sum of supporting evidence reliability

Contradicting Weight =

sum of contradicting evidence reliability

Therefore:

- Positive raw score → supporting evidence dominates.
- Negative raw score → contradicting evidence dominates.
- Zero raw score → no net evidence advantage.

7.5 Confidence Calculation

The implementation converts the raw score into a confidence percentage:

Confidence =

$$\frac{((\text{Raw Score} + \text{Total Weight}))}{(2 \times \text{Total Weight})} \times 100$$

where:

Total Weight =

Supporting Weight + Contradicting Weight

If no usable evidence exists, the implementation assigns:

Raw Score = 0

Confidence = 0%

This prevents unsupported confidence values.

7.6 Example

Suppose:

Supporting Weight = 0.8

Contradicting Weight = 0.2

Then:

Raw Score = 0.8 - 0.2

= 0.6

Total Weight = 1.0

Confidence =

$$\frac{((0.6 + 1.0) / 2) \times 100}{}$$

= 80%

Therefore, the hypothesis receives an 80% confidence value.

8. DECISION, SYNTHESIS AND DISCRIMINATING TESTS

8.1 Decision Layer

The decision layer converts confidence scores into an action.

The system supports:

SELECT

COMBINE

TEST

ABSTAIN

8.2 SELECT

A hypothesis can be selected when:

- It has sufficient confidence.
- It clearly leads over competing hypotheses.

For example, in the evaluation data, a recent deployment with direct evidence of a newly deployed version produced H2 as the selected hypothesis.

8.3 COMBINE

Sometimes two hypotheses can both have strong evidence.

If the hypotheses are compatible, the system can synthesize them.

Example:

H2: Recent software deployment

+

H1: Database failure

↓

COMBINED_H2_H1

The synthesized hypothesis combines:

- Assumptions

- Predicted effects
- Required evidence

from both original hypotheses.

The held-out evaluation includes such a case, where H2 and H1 both had strong evidence and were combined into a synthesized hypothesis.

8.4 TEST

If two hypotheses remain difficult to distinguish, the system recommends an additional diagnostic test.

Example:

Hypothesis A:

Recent deployment

Hypothesis B:

Database failure

A recommended test is to compare database behavior between the previous and current application versions.

The purpose is to determine whether the failure is independent of the application version or appears specifically with the newly deployed version.

8.5 ABSTAIN / UNKNOWN

The system does not force a known diagnosis when evidence is insufficient.

For example, DNS-related or authentication-service failures in the evaluation set did not produce sufficient evidence for H1–H5. The system therefore abstained.

This is an important property because an unsupported diagnosis can be more harmful than explicitly stating that the available information is insufficient.

9. INVESTIGATION BUDGET AND STOPPING

9.1 Investigation Budget

The system uses a bounded investigation process.

The budget tracks:

- Maximum number of hypotheses.
- Maximum investigation rounds.
- Maximum evidence checks.

The current configuration includes:

Maximum hypotheses: 6

Maximum rounds: 3

Maximum evidence checks: 30

This prevents uncontrolled exploration.

9.2 Stopping Conditions

Investigation can stop when:

Strong Hypothesis

A strong hypothesis clearly leads.

Insufficient Evidence

No known hypothesis has enough evidence.

Close Hypotheses

The strongest hypotheses remain sufficiently close that further reasoning should move toward synthesis or a discriminating test.

Budget Exhaustion

The investigation has reached its allowed exploration limit.

9.3 Importance of Stopping

A diagnostic system should not continue collecting or processing evidence indefinitely.

Stopping rules provide a clear boundary between:

Investigation



Sufficient information



Decision

This makes the diagnostic process controlled and measurable.

10. IMPLEMENTATION AND TECHNOLOGIES

10.1 Programming Language

The system is implemented in **Python**.

Python was selected because it provides:

- Simple data modelling.
- Modular development.
- Easy testing.
- Convenient data processing.
- Rapid prototyping.

10.2 Main Technologies

Technology Purpose

Python	Core implementation
pytest	Automated testing
JSON	Evaluation and evidence datasets
Dataclasses	Structured hypothesis/evidence models
VS Code	Development environment
Git	Version control
GitHub	Source-code repository

10.3 Project Structure

Software_Failure_Diagnosis_AI/

|

```
└─ data/
|   └─ evidence.json
|   └─ failures.json
|   └─ test_cases.json
|   └─ held_out_cases.json
|   └─ evaluation_results.json
|
└─ docs/
|   └─ architecture.md
|   └─ hypothesis_design.md
|   └─ problem_definition.md
|   └─ scoring_method.md
|
└─ src/
|   └─ decision.py
|   └─ distinctness.py
|   └─ divergence.py
|   └─ evidence.py
|   └─ hypothesis.py
|   └─ main.py
|   └─ scoring.py
|   └─ test_recommender.py
|   └─ budget.py
|   └─ stopping.py
|   └─ synthesis.py
|   └─ hypothesis_validator.py
|   └─ ai_hypothesis.py
```

```
|
|└─ tests/
|  |└─ test_divergence.py
|  |└─ test_duplicates.py
|  |└─ test_ties.py
|  |└─ test_unknown.py
|  |└─ test_evidence.py
|  |└─ test_dependent_evidence.py
|  |└─ test_scoring.py
|  |└─ test_confidence_update.py
|  |└─ test_recommender.py
|  |└─ test_decision.py
|  |└─ test_combine.py
|  |└─ test_budget.py
|  |└─ test_stopping.py
|  |└─ test_end_to_end.py
|  |└─ test_evaluation.py
|  |└─ test_calibration.py
|  |└─ test_held_out.py
|  |└─ test_hypothesis_validator.py
|  |└─ test_synthesis.py
|  |└─ test_incompatible_synthesis.py
|  |└─ test_full_system.py
|  |└─ test_combine_decision.py
|  └─ test_end_to_end_combine.py
|
|└─ README.md
```

└─ requirements.txt

10.4 Local Execution

The core diagnostic pipeline is designed to operate locally.

The project does not require paid API usage for its core diagnostic workflow. The local hypothesis generator provides deterministic candidate hypotheses for the implemented diagnostic pipeline.

11. TESTING AND EVALUATION

11.1 Unit and Integration Testing

The project uses pytest for automated testing.

The test suite covers:

- Hypothesis divergence.
- Duplicate detection.
- Hypothesis distinctness.
- Evidence assessment.
- Dependent evidence.
- Confidence scoring.
- Confidence updates.
- Decision making.
- Discriminating test recommendation.
- Investigation budget.
- Stopping rules.
- Hypothesis synthesis.
- Incompatible synthesis.
- End-to-end diagnosis.
- Evaluation.
- Calibration.
- Held-out cases.

The final automated test run produced:

12 passed

11.2 Test Case Evaluation

The primary evaluation dataset contains **12 test cases** covering known failure types and unknown/insufficient-evidence situations.

The cases include:

- Database failure.
- Recent deployment.
- Server/resource problem.
- External API failure.
- Configuration problem.
- Ambiguous/close hypotheses.
- Unknown causes.
- Insufficient diagnostic information.

The final evaluation produced:

Metric	Result
Total test cases	12
Correct cases	12
Accuracy	100.00%
Abstained cases	3
Abstention rate	25.00%
Selective accuracy	100.00%
Brier score	0.3056

These results are from the current project evaluation dataset.

11.3 Held-Out Evaluation

A separate set of six held-out cases was evaluated.

The held-out cases covered:

- Database failure.
- Deployment-related failure.
- Server/resource problem.
- External API failure.
- Configuration problem.
- Unknown cause.

The system produced:

Metric	Result
Total held-out cases	6
Correct	6
Accuracy	100.00%

The held-out results therefore show that all six cases in this evaluation set were correctly handled.

11.4 Unknown-Cause Testing

Unknown cases are particularly important because the system should not force an incorrect known diagnosis.

For cases involving insufficient information, the system produced:

Outcome: ABSTAIN

Selected: []

The evaluation treated these abstentions as correct handling of the corresponding UNKNOWN cases.

12. RESULTS AND DISCUSSION

12.1 Overall Results

The current evaluation demonstrates that the implemented diagnostic pipeline correctly handled all 12 cases in the primary evaluation dataset.

12 / 12 correct

100% accuracy

The system also correctly abstained on three cases where the predefined hypothesis space was not sufficiently supported by the available evidence.

12.2 Selective Diagnosis

The system demonstrated the ability to distinguish between cases where a diagnosis can be supported and cases where it should abstain.

This is important because the objective is not simply to maximize the number of predictions.

A diagnostic system should also recognize when its evidence is inadequate.

The evaluation produced:

Selective accuracy: 100%

on the current test dataset.

12.3 Hypothesis Selection

Strong evidence resulted in direct hypothesis selection.

For example, extremely high server CPU and memory usage supported H3, resulting in H3 being selected in the evaluation.

Similarly, an external API health-check failure supported H4 in the evaluation cases.

12.4 Hypothesis Synthesis

The system also demonstrated that diagnosis does not always need to be limited to one cause.

In the held-out deployment/database case, both H2 and H1 had strong supporting evidence. The system therefore generated:

COMBINED_H2_H1

as a synthesized hypothesis.

This allows the system to represent situations where multiple factors may contribute to a failure.

12.5 Discriminating Tests

When hypotheses remain close, the system can recommend an additional diagnostic test.

For example, the deployment-versus-database scenario generated a test comparing database behavior between the previous and current application versions.

This provides a practical next step instead of simply returning an ambiguous prediction.

12.6 Interpretation of Results

The 100% accuracy reported here should be interpreted within the scope of the current evaluation datasets.

The datasets contain a limited set of constructed diagnostic scenarios and do not represent the full diversity of real-world software failures.

Therefore, the results demonstrate that the implemented reasoning pipeline behaves correctly on the selected evaluation cases, rather than establishing 100% real-world diagnostic accuracy.

13. LIMITATIONS AND FUTURE SCOPE

13.1 Limitations

Limited Hypothesis Space

The current local diagnostic engine focuses on five major known causes:

- Database failure.
- Recent deployment.
- Server/resource problem.
- External API failure.
- Configuration/environment problem.

Real software systems can fail for many additional reasons.

Rule-Based Evidence Assessment

The current evidence assessment uses predefined patterns and keywords.

Therefore, complex natural-language evidence may not always be interpreted correctly.

Limited Dataset Size

The current evaluation uses 12 primary cases and 6 held-out cases.

A larger and more diverse dataset would provide stronger evidence of generalization.

Simplified Compatibility Rules

Hypothesis compatibility is determined using predefined rules.

Real-world causal relationships can be more complex.

Local Deterministic Generation

The current core workflow uses a local deterministic hypothesis generator.

A more advanced system could incorporate additional knowledge sources or language models while retaining the same validation and uncertainty controls.

13.2 Future Scope

Larger Hypothesis Library

Expand the hypothesis space to include:

- Network failures.
- Authentication failures.
- DNS failures.
- Storage failures.
- Security-related failures.
- Dependency failures.
- Infrastructure failures.

Log Analysis

Integrate application logs, server logs, database logs, and monitoring metrics.

Natural Language Processing

Use NLP to extract structured evidence from unstructured incident descriptions.

Real-Time Monitoring

Connect the system with monitoring systems to continuously collect diagnostic evidence.

Adaptive Test Selection

Automatically select the next diagnostic test based on expected information gain.

Larger Evaluation Dataset

Evaluate the system using a much larger collection of real and synthetic software incidents.

Advanced Calibration

Improve confidence calibration using larger validation datasets.

Human-in-the-Loop Diagnosis

Allow software engineers to review, accept, reject, or add evidence to hypotheses.

14. CONCLUSION

Software Failure Diagnosis AI presents a structured approach to diagnosing software failures under uncertainty.

The system does not rely solely on selecting a single predicted cause. Instead, it generates multiple hypotheses, validates their distinctness, evaluates evidence, calculates confidence, handles dependent evidence, and explicitly represents uncertainty.

The system supports four important diagnostic outcomes:

SELECT

COMBINE

TEST

ABSTAIN

This allows the diagnostic process to adapt to different evidence conditions.

The inclusion of an explicit UNKNOWN/abstention mechanism prevents the system from forcing unsupported diagnoses. The discriminating-test mechanism provides a practical next step when competing hypotheses remain close. Hypothesis synthesis allows compatible causes to be represented together.

The investigation budget and stopping rule provide controlled exploration.

In the current evaluation, the system correctly handled all 12 primary test cases and all 6 held-out cases. The results demonstrate the functionality of the implemented diagnostic pipeline on the selected evaluation datasets.

Although the current implementation has limitations in hypothesis coverage, evidence interpretation, and dataset size, it provides a foundation for a more advanced software incident diagnosis system incorporating logs, monitoring data, NLP, larger knowledge bases, and adaptive diagnostic testing.

REFERENCES

Use this section at the end of the Word document.

1. Python Software Foundation, **Python Documentation**.
2. pytest Development Team, **pytest Documentation**.
3. Git Documentation, **Git Reference Documentation**.
4. GitHub, **GitHub Documentation**.
5. Project source code: **Software Failure Diagnosis AI**, GitHub repository.
6. Project documentation:
 - docs/problem_definition.md
 - docs/architecture.md
 - docs/hypothesis_design.md
 - docs/scoring_method.md
7. Project evaluation datasets:
 - data/test_cases.json
 - data/held_out_cases.json
 - data/evaluation_results.json

APPENDIX A — SAMPLE DIAGNOSTIC FLOW

You can add this after References if you need a little more content/pages.

Example: Database Failure

Problem:

The web application suddenly started returning HTTP 500 errors.

Evidence:

Database connection timeout detected

Database health check is normal

Assessment:

H1 → Supporting + Contradicting

H2 → Missing

H3 → Missing

H4 → Missing

H5 → Contradicting

Decision:

H1 → SELECT

The evaluation output shows H1 receiving the strongest evidence in the corresponding database case.

APPENDIX B — SAMPLE UNKNOWN CASE

Problem:

The application is unavailable because of a network name-resolution problem.

Evidence:

DNS resolution is failing

DNS service is unavailable

The current hypothesis set does not contain a dedicated DNS hypothesis.

Therefore:

H1 → 0%

H2 → 0%

H3 → 0%

H4 → 0%

H5 → 0%

Decision → ABSTAIN / UNKNOWN

This demonstrates that the system does not force an unrelated known hypothesis when the observed failure falls outside its current hypothesis space.

APPENDIX C — FINAL TEST STATUS

Automated Tests

12 passed

Primary Evaluation

Total Cases : 12

Correct Cases : 12

Accuracy : 100.00%

Abstention Rate : 25.00%

Selective Accuracy: 100.00%

Brier Score : 0.3056

Held-Out Evaluation

Total Cases : 6

Correct Cases : 6

Accuracy : 100.00%

These figures correspond to the current project evaluation outputs.